

COMPANY PROJECT PRESENTATION

AI TestFixer

An Intelligent Agent-Based Self-Healing
Framework for Selenium Test Automation

Python | Selenium WebDriver | Ollama (Llama 3) | SQLite | Streamlit | GitHub

ABSTRACT

AI TestFixer is an intelligent, agent-based self-healing framework designed to automatically detect and repair broken Selenium test scripts caused by UI changes in web applications.

The Problem: When developers change the UI (button IDs, class names, XPaths), automated Selenium tests break — requiring manual QA intervention that wastes time and resources.

The Solution: AI TestFixer uses **Ollama (Llama 3)** running locally to analyze failure logs, identify changed locators, and automatically suggest and apply fixes — all stored in **SQLite** and visualized through a **Streamlit** dashboard.

PROBLEM STATEMENT



Frequent Test Breakage

UI changes (IDs, classes, XPaths) cause Selenium tests to fail constantly



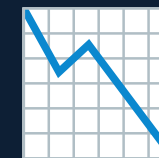
Manual Fix Time

QA engineers spend hours manually finding and fixing broken locators



High Maintenance Cost

Continuous human intervention increases project cost and delays releases



Low Test Reliability

Teams lose trust in automated tests due to frequent false failures

SOLUTION

AI TestFixer — The Self-Healing Approach



Detect

Selenium test fails
& logs error



Analyze

Ollama AI reads
error + DOM



Fix

AI suggests
new locator



Validate

Test re-runs
& passes ✓

Zero manual intervention — AI fixes tests automatically, logs results in SQLite, and displays on Streamlit dashboard

OBJECTIVES

01 Build a self-healing Selenium test framework using Python that detects test failures automatically

02 Integrate Ollama (Llama 3) locally to analyze test logs and suggest accurate locator fixes without API costs

03 Store all test history, failed locators, and AI-suggested fixes in a SQLite database for tracking

04 Develop a Streamlit dashboard to visualize test results, healing actions, and execution reports

05 Reduce manual QA intervention time and improve overall test suite reliability and maintainability

RECENT NEWS & STATISTICS

72%

of teams report
locator failures

40%

of QA time spent
fixing broken tests

3x

faster releases with
self-healing tests

60%

reduction in test
maintenance cost

Industry News:

- ▶ Gartner (2024): AI-augmented test automation is a top priority for 68% of software teams
- ▶ GitHub & Atlassian report increasing adoption of LLM-based test repair tools in CI/CD pipelines
- ▶ Selenium 4 + AI integration is the fastest growing area in test automation (StackOverflow Survey 2024)

RELATED WORKS & LITERATURE SURVEY

Visual Healing

WATERFALL (2019)

Automated locator repair using visual DOM similarity — limited to image-based matching

ML-Based

HEALENIUM (2021)

Open-source self-healing Selenium library using ML tree-based locator prediction

Cloud AI

APPLITOLS (2022)

AI-powered visual testing with self-healing — cloud-based, expensive for enterprises

Proprietary

TESTIM AI (2023)

AI test authoring + self-healing but requires proprietary platform lock-in

✓ This Project

OUR APPROACH (2024)

Local LLM (Ollama Llama 3) + Selenium + SQLite — free, open, customizable

EXISTING METHODS

Method	Pros 	Cons 
Manual Locator Fix	Simple, accurate	Time-consuming, human error prone
Rule-Based Healing	Fast execution	Brittle, fails on complex UI changes
Visual AI Testing	Screenshot-based	Cloud dependency, high cost

PROPOSED METHODOLOGY

1

Test Execution

Selenium runs tests via Python + Pytest on target web app

2

Failure Detection

Exception caught → Screenshot + Error log captured automatically

3

AI Analysis

Ollama Llama 3 analyzes DOM + error log to identify locator change

4

Fix Generation

AI generates new XPath/CSS selector → stored in SQLite

5

Auto Repair

Test re-runs with new locator → Pass/Fail logged

6

Dashboard

Streamlit shows real-time healing reports & test analytics

TOOLS AND TECHNOLOGY



Python

Core Language

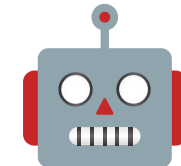
Easy development, rich AI + Selenium library support



Selenium WebDriver

Test Automation

Industry-standard browser automation, provides failure data



Ollama (Llama 3)

AI Engine

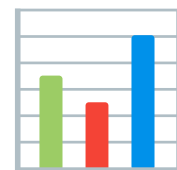
Free, local LLM — analyzes errors, suggests locator fixes



SQLite

Database

Lightweight DB — stores test history, failures, and fixes



Streamlit

Dashboard

Interactive web UI — shows results, healing logs, reports



GitHub

Version Control

Code management, team collaboration, judge showcase

TARGET AUDIENCE



QA Engineers

Reduce manual test maintenance time and focus on higher-value testing activities



Software Developers

Ensure tests remain stable even after frequent UI changes during development



Agile / DevOps Teams

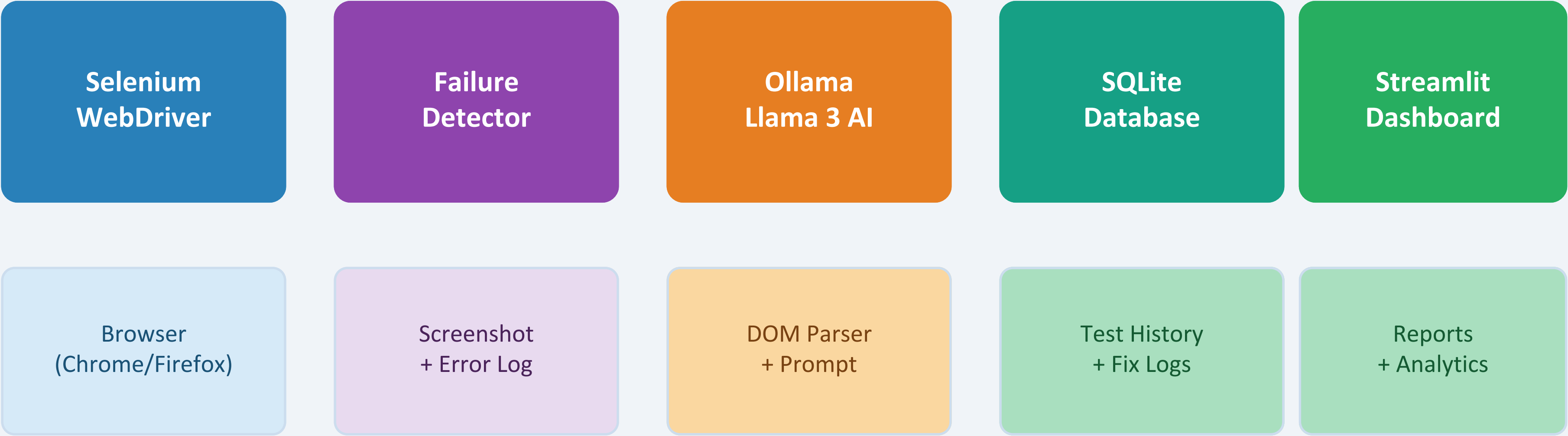
Integrate self-healing tests into CI/CD pipelines for faster, reliable releases



Startups & Enterprises

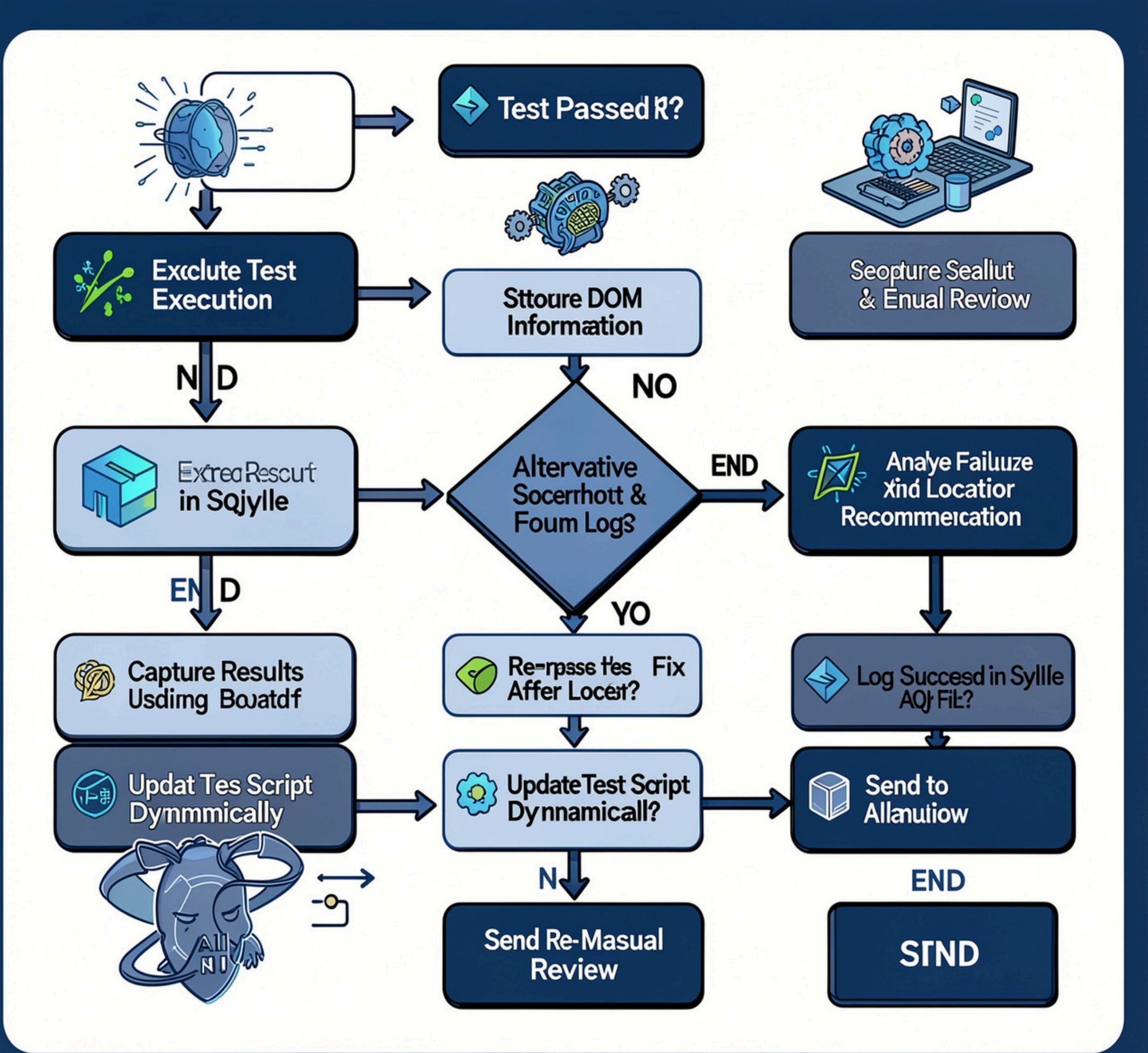
Cut QA costs while maintaining high-quality automated test suites at scale

ARCHITECTURE DIAGRAM



← All components run locally — No cloud dependency | Python orchestrates entire pipeline →

FLOWCHART



ADVANTAGES



Zero Cost AI

Ollama runs Llama 3 locally — no API fees, no cloud subscription required



Fully Automated

Tests self-heal without any human intervention after initial setup



Visual Dashboard

Streamlit provides real-time test analytics, healing logs, and failure trends



Data Privacy

Everything runs locally — test data never leaves your machine



Easy Integration

Works with any existing Selenium + Python test suite with minimal changes



Scalable

SQLite stores complete history; scales with growing test suites over time

LIMITATIONS & DRAWBACKS



Local Hardware Dependency

Ollama requires sufficient RAM/CPU to run Llama 3 effectively (8GB+ RAM recommended)



AI Accuracy Limitations

LLM suggestions may not always be 100% accurate — complex DOM changes may need manual review



Web Application Scope

Currently focused on web UI testing only — not applicable to API or mobile test automation



Learning Curve

Teams unfamiliar with AI/LLM concepts may need initial training to use and maintain the system



Dynamic Content

Highly dynamic UIs (e.g., infinite scrolling, canvas-based) may not be reliably healed

FUTURE ENHANCEMENT

Phase 1



Mobile Testing Support

Extend self-healing to Appium for Android/iOS mobile test automation

Phase 2



CI/CD Integration

Jenkins & GitHub Actions plugin for automatic healing in deployment pipelines

Phase 3



Fine-tuned AI Model

Train a custom model specifically on Selenium patterns for higher fix accuracy

Phase 4



Cloud Dashboard

Multi-user Streamlit cloud deployment with team collaboration features

Thank You!

AI TestFixer — Self-Healing Selenium Test Automation

Built with:

Python • Selenium WebDriver • Ollama (Llama 3) • SQLite • Streamlit • GitHub

Questions & Discussion